

Banking_Analysis

December 7, 2025

1 Comprehensive Banking Data Analysis

This notebook provides a detailed analysis of the Portuguese banking institution's direct marketing campaigns. It specifically addresses the 18 questions outlined in the problem statement.

```
[23]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set visual style
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

# Load Data
df = pd.read_csv('banking_data.csv')
print(f"Dataset Shape: {df.shape}")
df.head()
```

Dataset Shape: (45216, 19)

```
[23]:
```

	age	job	marital	marital_status	education	default	balance	\
0	58	management	married	married	tertiary	no	2143	
1	44	technician	single	single	secondary	no	29	
2	33	entrepreneur	married	married	secondary	no	2	
3	47	blue-collar	married	married	unknown	no	1506	
4	33	unknown	single	single	unknown	no	1	

	housing	loan	contact	day	month	day_month	duration	campaign	pdays	\
0	yes	no	unknown	5	may	5-May	261	1	-1	
1	yes	no	unknown	5	may	5-May	151	1	-1	
2	yes	yes	unknown	5	may	5-May	76	1	-1	
3	yes	no	unknown	5	may	5-May	92	1	-1	
4	no	no	unknown	5	may	5-May	198	1	-1	

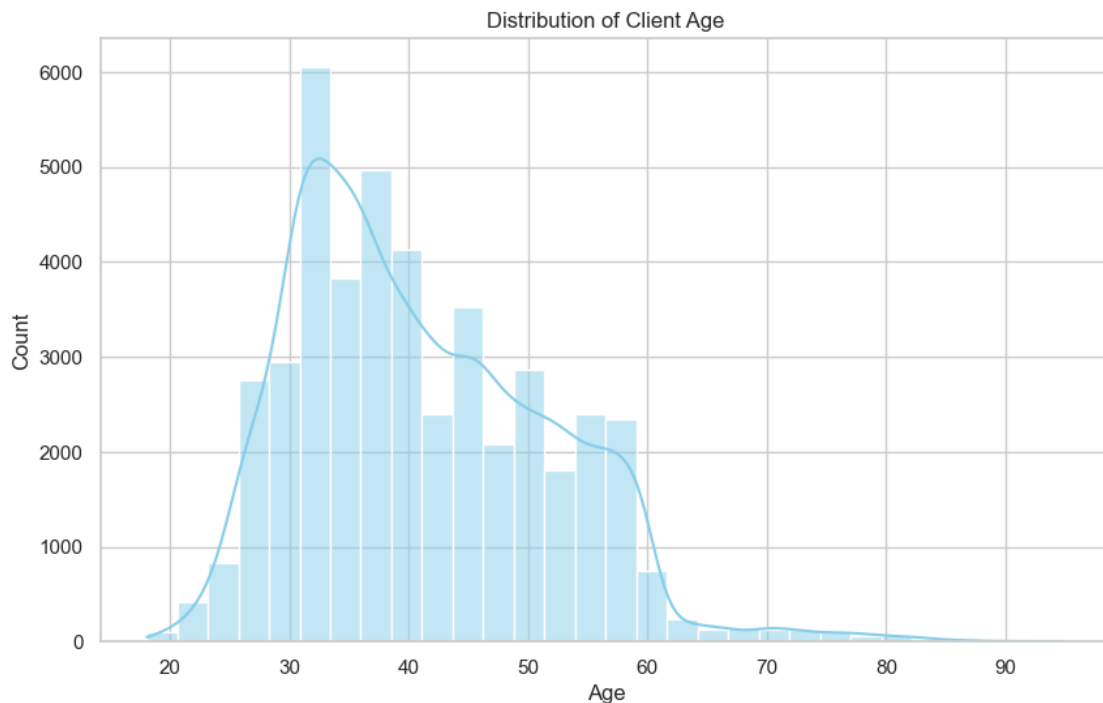
	previous	poutcome	y
0	0	unknown	no
1	0	unknown	no

```
2      0  unknown  no
3      0  unknown  no
4      0  unknown  no
```

1.1 1. What is the distribution of age among the clients?

```
[24]: plt.figure(figsize=(10, 6))
sns.histplot(df['age'], bins=30, kde=True, color='skyblue')
plt.title('Distribution of Client Age')
plt.xlabel('Age')
plt.show()

print(df['age'].describe())
```



```
count    45216.000000
mean      40.938186
std       10.621249
min       18.000000
25%      33.000000
50%      39.000000
75%      48.000000
max       95.000000
Name: age, dtype: float64
```

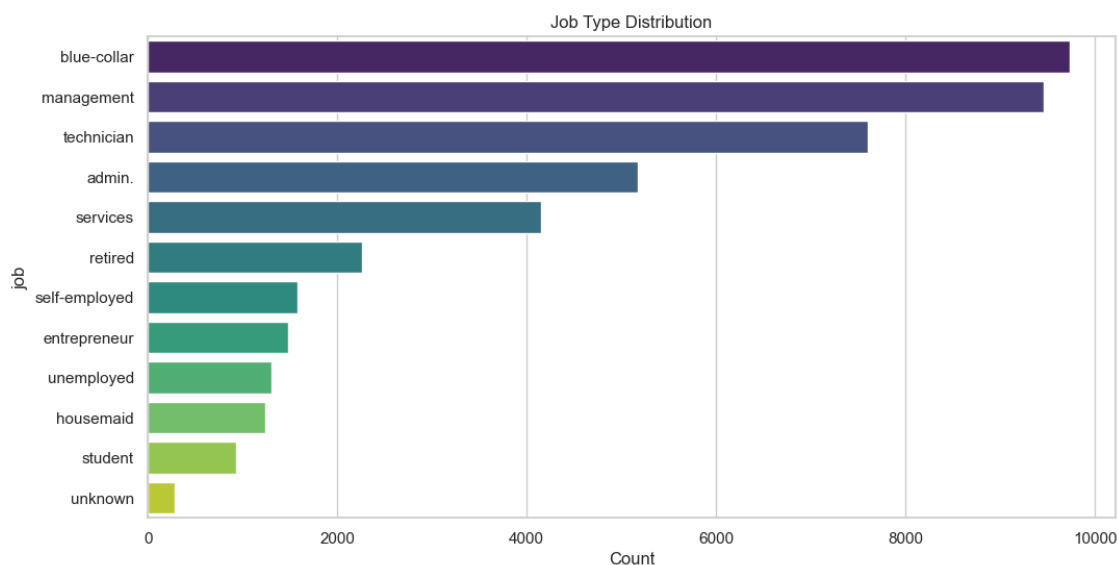
1.2 2. How does the job type vary among the clients?

```
[25]: plt.figure(figsize=(12, 6))
sns.countplot(data=df, y='job', order=df['job'].value_counts().index,
             palette='viridis')
plt.title('Job Type Distribution')
plt.xlabel('Count')
plt.show()
```

/var/folders/h4/x04vtstr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/2897975305.py:2
: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, y='job', order=df['job'].value_counts().index,
             palette='viridis')
```



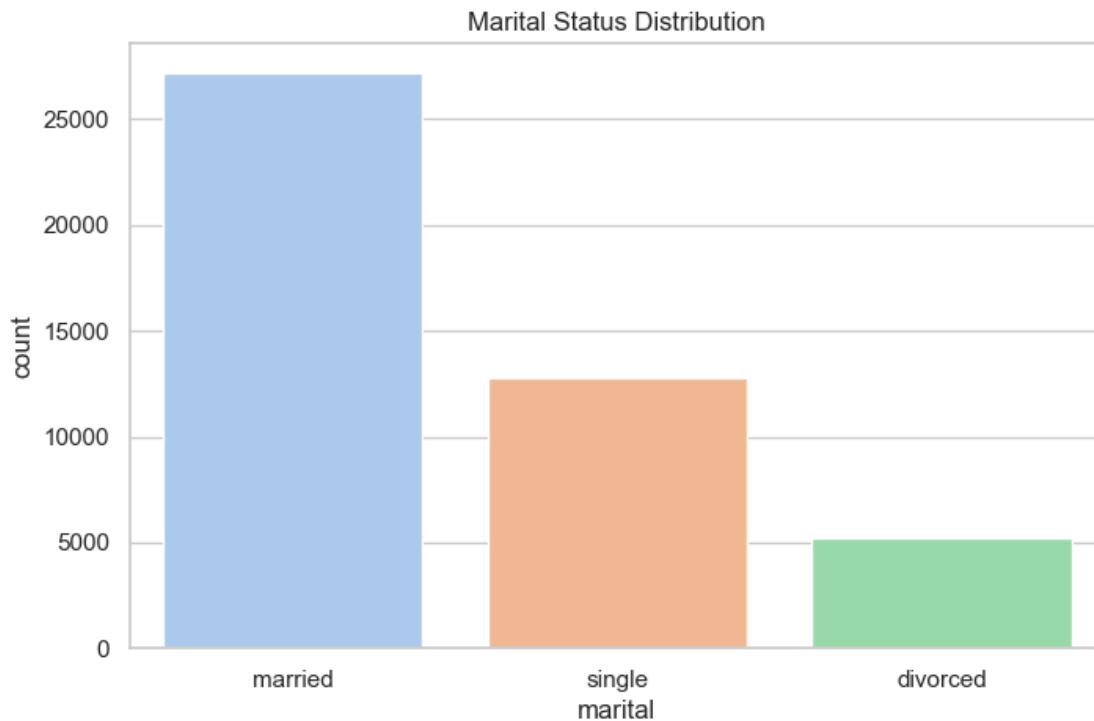
1.3 3. What is the marital status distribution of the clients?

```
[26]: plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='marital', palette='pastel')
plt.title('Marital Status Distribution')
plt.show()
```

/var/folders/h4/x04vtstr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/753160876.py:2:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='marital', palette='pastel')
```



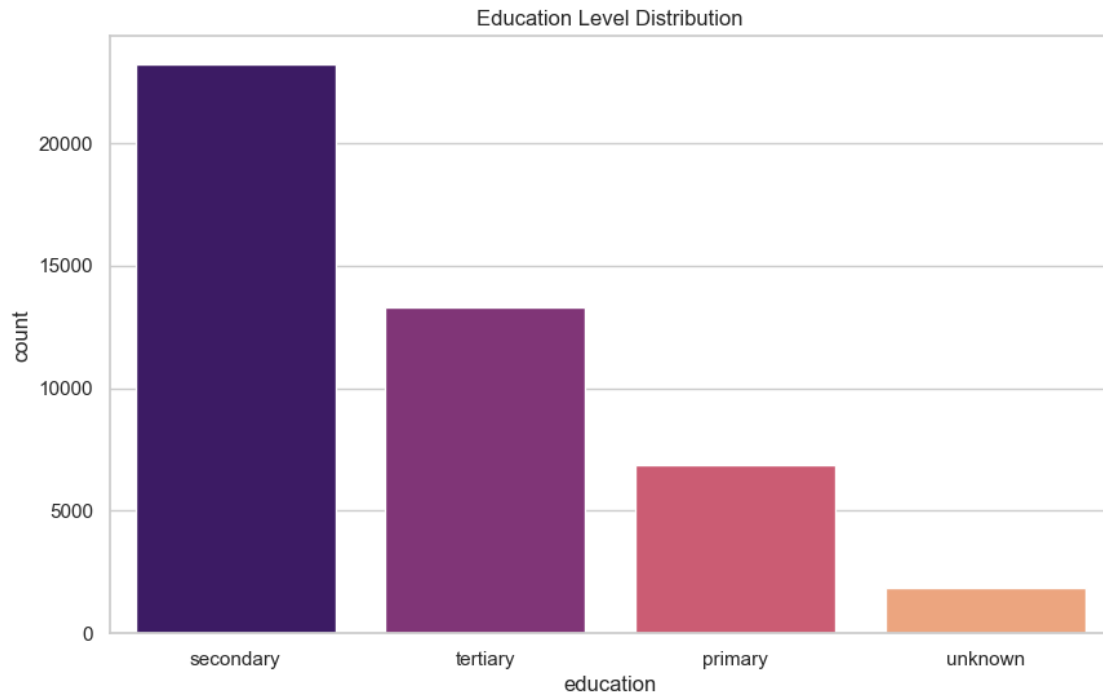
1.4 4. What is the level of education among the clients?

```
[27]: plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='education', order=df['education'].value_counts().
    ↪index, palette='magma')
plt.title('Education Level Distribution')
plt.show()
```

```
/var/folders/h4/x04vtshr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/326421928.py:2:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

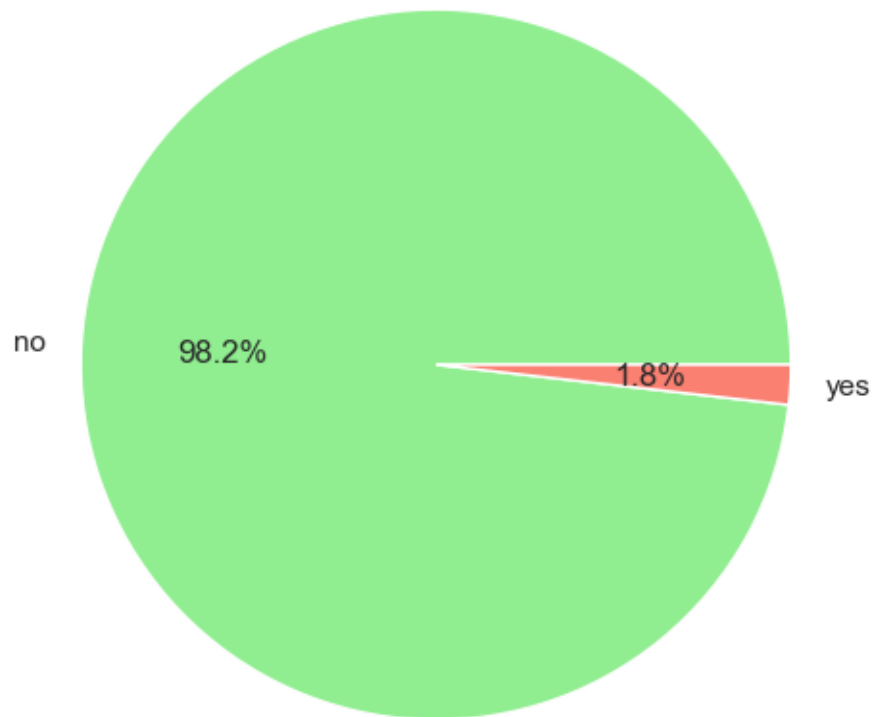
```
sns.countplot(data=df, x='education',
order=df['education'].value_counts().index, palette='magma')
```



1.5 5. What proportion of clients have credit in default?

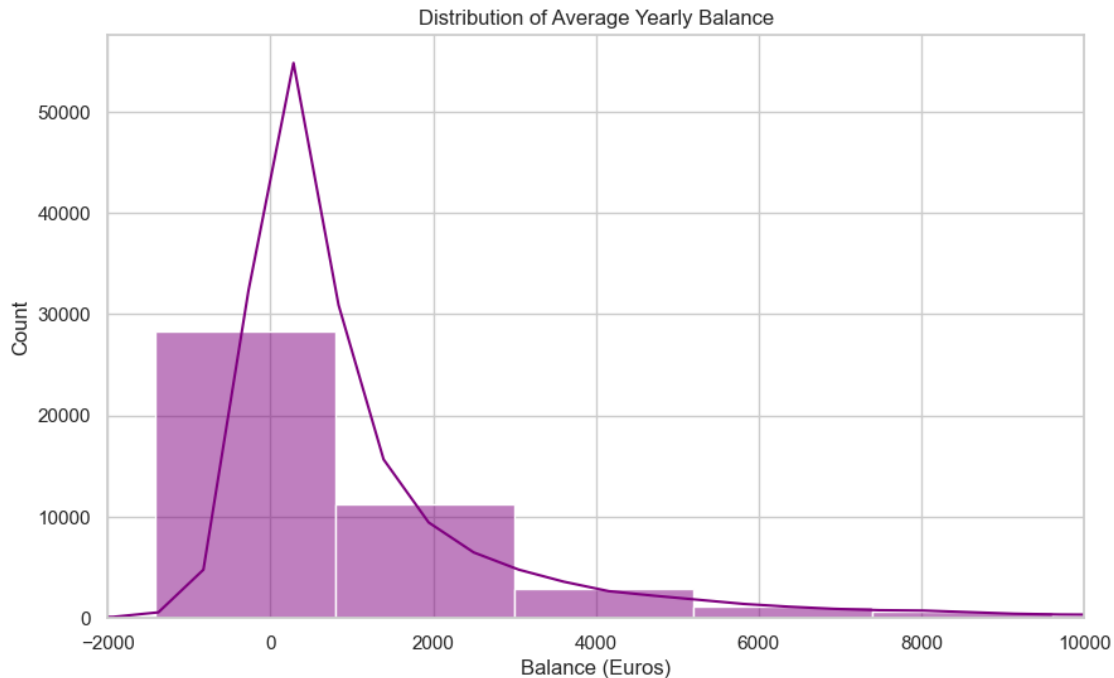
```
[28]: plt.figure(figsize=(6, 6))
df['default'].value_counts().plot.pie(autopct='%1.1f%%', colors=['lightgreen', 'salmon'])
plt.title('Proportion of Credit in Default')
plt.ylabel('')
plt.show()
```

Proportion of Credit in Default



1.6 6. What is the distribution of the average yearly balance among the clients?

```
[29]: plt.figure(figsize=(10, 6))
sns.histplot(df['balance'], bins=50, kde=True, color='purple')
plt.title('Distribution of Average Yearly Balance')
plt.xlabel('Balance (Euros)')
plt.xlim(-2000, 10000) # Limiting x-axis to see the bulk of data better
plt.show()
print(df['balance'].describe())
```



```
count    45216.000000
mean      1362.277844
std       3044.609674
min       -8019.000000
25%        72.000000
50%       448.500000
75%      1428.000000
max     102127.000000
Name: balance, dtype: float64
```

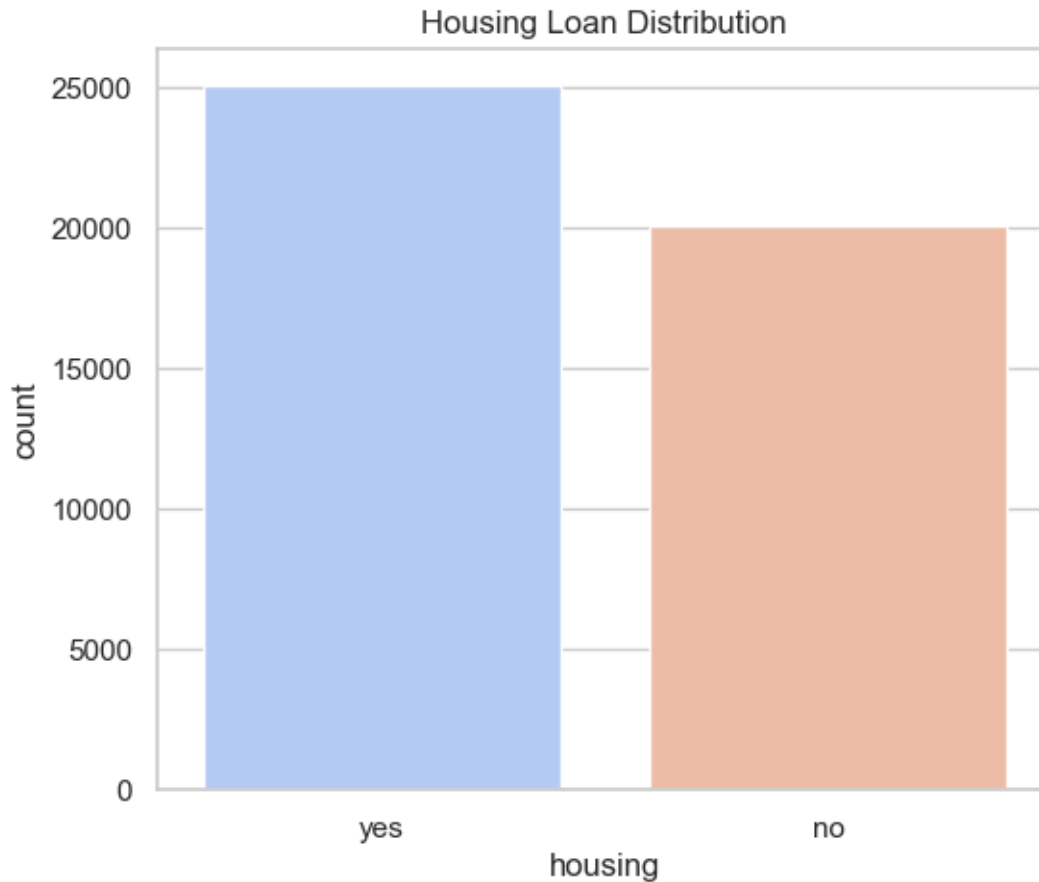
1.7 7. How many clients have housing loans?

```
[30]: plt.figure(figsize=(6, 5))
sns.countplot(data=df, x='housing', palette='coolwarm')
plt.title('Housing Loan Distribution')
plt.show()
```

/var/folders/h4/x04vtshr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/3225124075.py:2
: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='housing', palette='coolwarm')
```



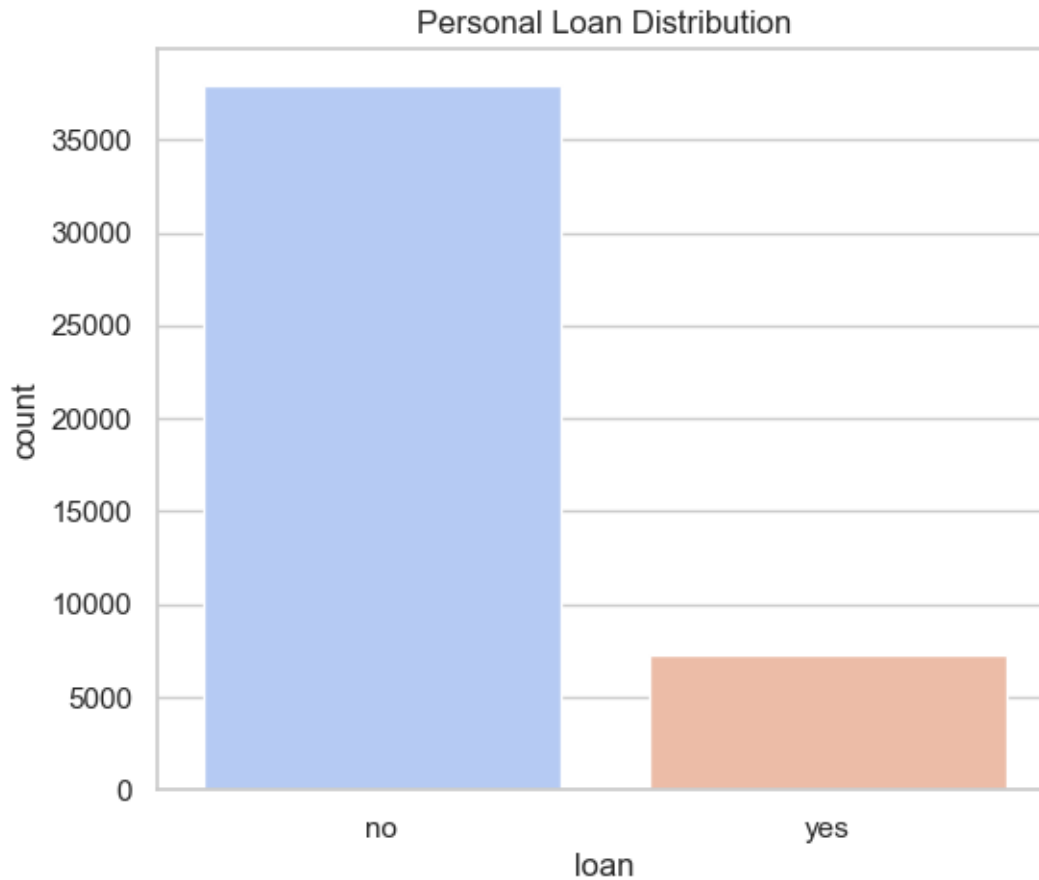
1.8 8. How many clients have personal loans?

```
[31]: plt.figure(figsize=(6, 5))
sns.countplot(data=df, x='loan', palette='coolwarm')
plt.title('Personal Loan Distribution')
plt.show()
```

/var/folders/h4/x04vtshr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/1090611000.py:2
: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='loan', palette='coolwarm')
```

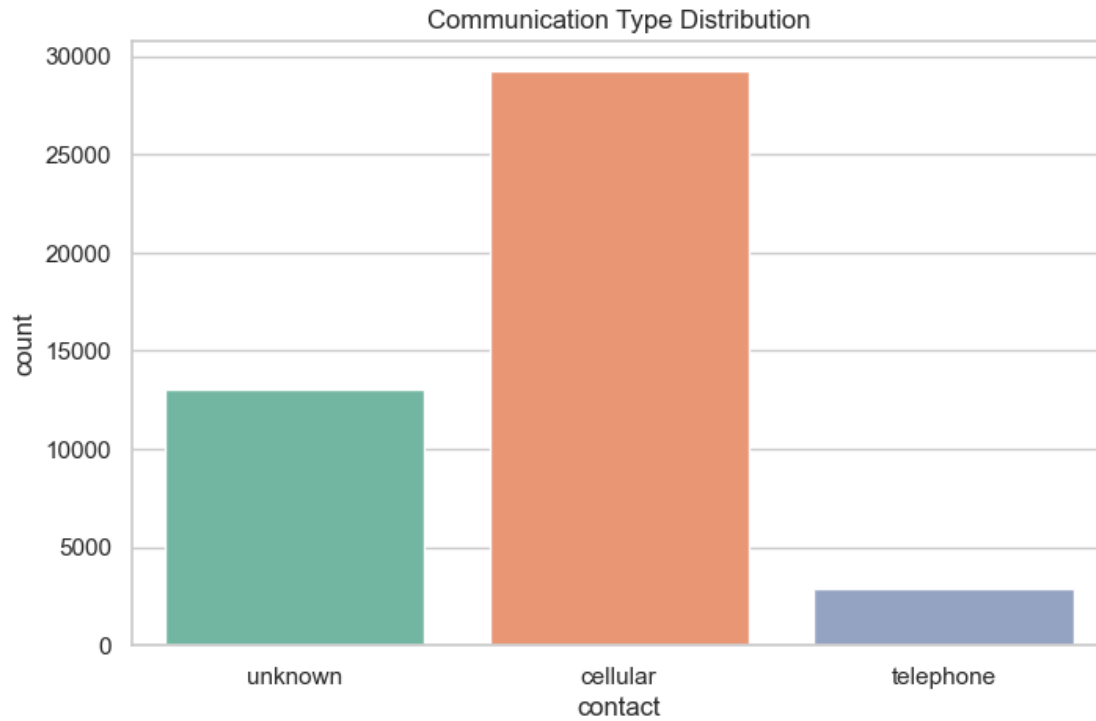
1.9 9. What are the communication types used for contacting clients?

```
[32]: plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='contact', palette='Set2')
plt.title('Communication Type Distribution')
plt.show()
```

/var/folders/h4/x04vtshr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/1732721413.py:2
: FutureWarning:

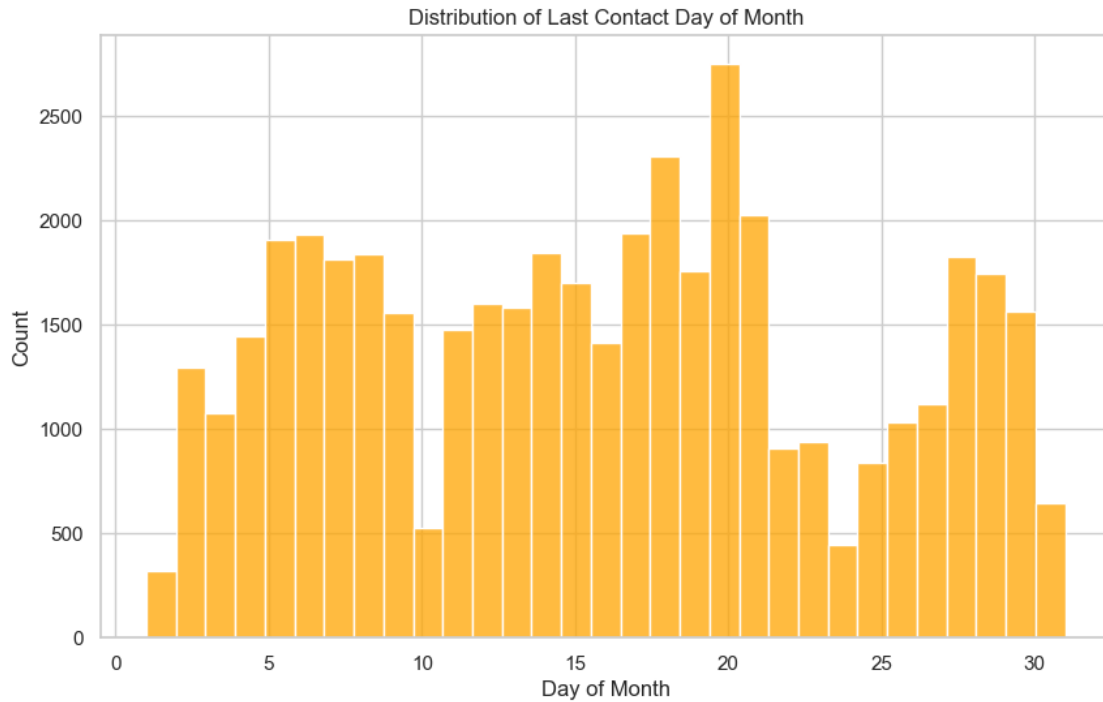
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='contact', palette='Set2')
```



1.10 10. What is the distribution of the last contact day of the month?

```
[33]: plt.figure(figsize=(10, 6))
sns.histplot(df['day'], bins=31, kde=False, color='orange')
plt.title('Distribution of Last Contact Day of Month')
plt.xlabel('Day of Month')
plt.show()
```



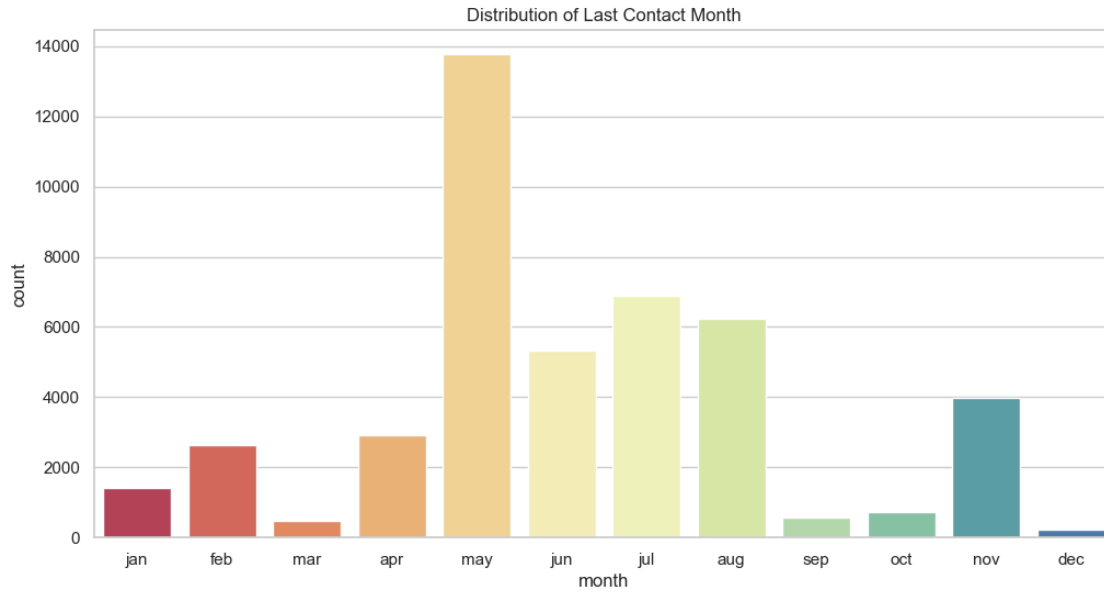
1.11 11. How does the last contact month vary among the clients?

```
[34]: months_order = ['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec']
plt.figure(figsize=(12, 6))
sns.countplot(data=df, x='month', order=months_order, palette='Spectral')
plt.title('Distribution of Last Contact Month')
plt.show()
```

/var/folders/h4/x04vtshr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/1200491370.py:3
: FutureWarning:

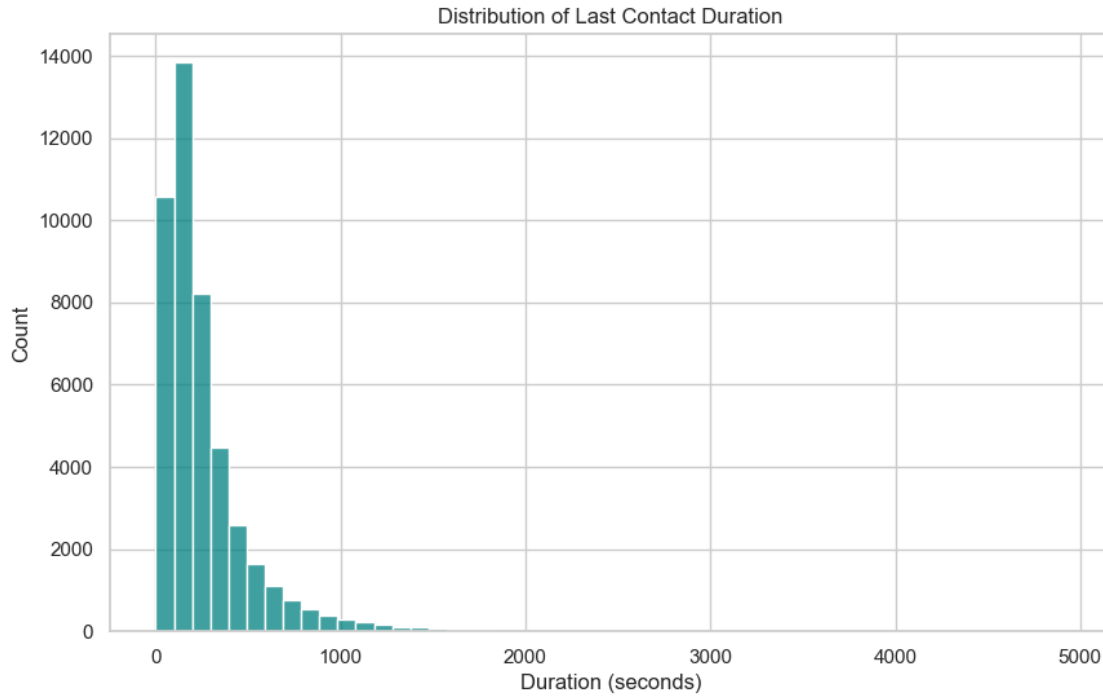
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='month', order=months_order, palette='Spectral')
```



1.12 12. What is the distribution of the duration of the last contact?

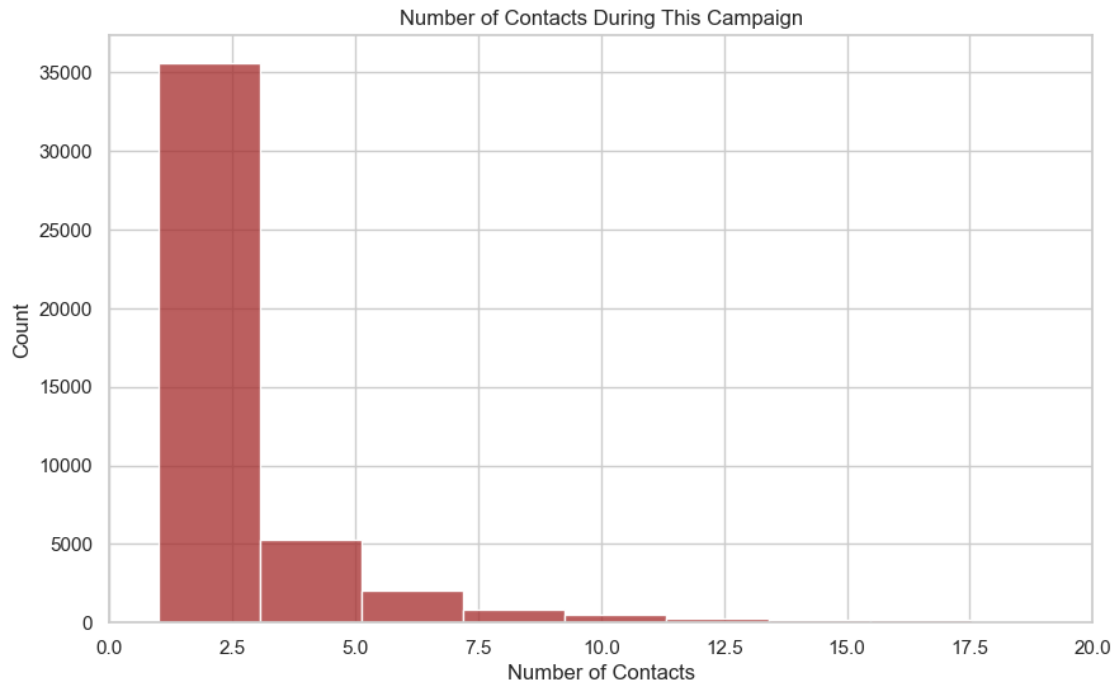
```
[35]: plt.figure(figsize=(10, 6))
sns.histplot(df['duration'], bins=50, color='teal')
plt.title('Distribution of Last Contact Duration')
plt.xlabel('Duration (seconds)')
plt.show()
print(df['duration'].describe())
```



```
count    45216.000000
mean      258.166202
std       257.515482
min         0.000000
25%       103.000000
50%       180.000000
75%       319.000000
max       4918.000000
Name: duration, dtype: float64
```

1.13 13. How many contacts were performed during the campaign for each client?

```
[36]: plt.figure(figsize=(10, 6))
sns.histplot(df['campaign'], bins=30, kde=False, color='brown')
plt.title('Number of Contacts During This Campaign')
plt.xlabel('Number of Contacts')
plt.xlim(0, 20) # Zooming in on the majority
plt.show()
```

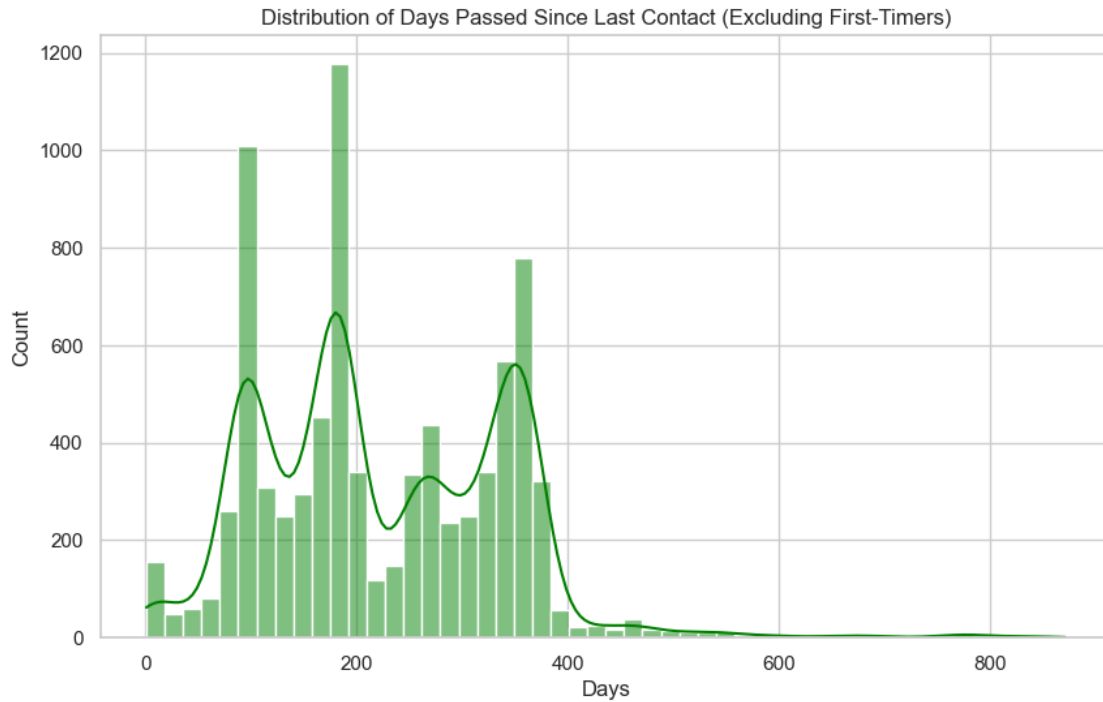


1.14 14. What is the distribution of 'pdays'?

```
[37]: # pdays = -1 means client was not previously contacted
contacted_previously = df[df['pdays'] != -1]

plt.figure(figsize=(10, 6))
sns.histplot(contacted_previously['pdays'], bins=50, kde=True, color='green')
plt.title('Distribution of Days Passed Since Last Contact (Excluding_
↳First-Timers)')
plt.xlabel('Days')
plt.show()

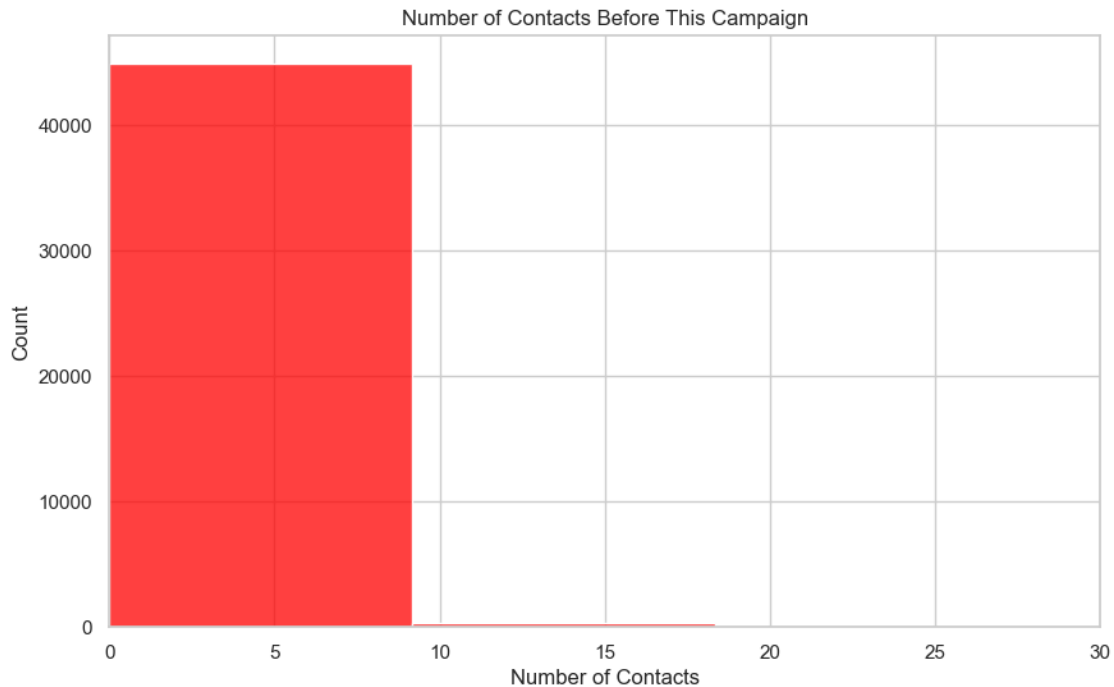
print(f"Clients contacted previously: {len(contacted_previously)} out of_
↳{len(df)}")
```



Clients contacted previously: 8260 out of 45216

1.15 15. How many contacts were performed before the current campaign for each client?

```
[38]: plt.figure(figsize=(10, 6))
sns.histplot(df['previous'], bins=30, kde=False, color='red')
plt.title('Number of Contacts Before This Campaign')
plt.xlabel('Number of Contacts')
plt.xlim(0, 30)
plt.show()
```



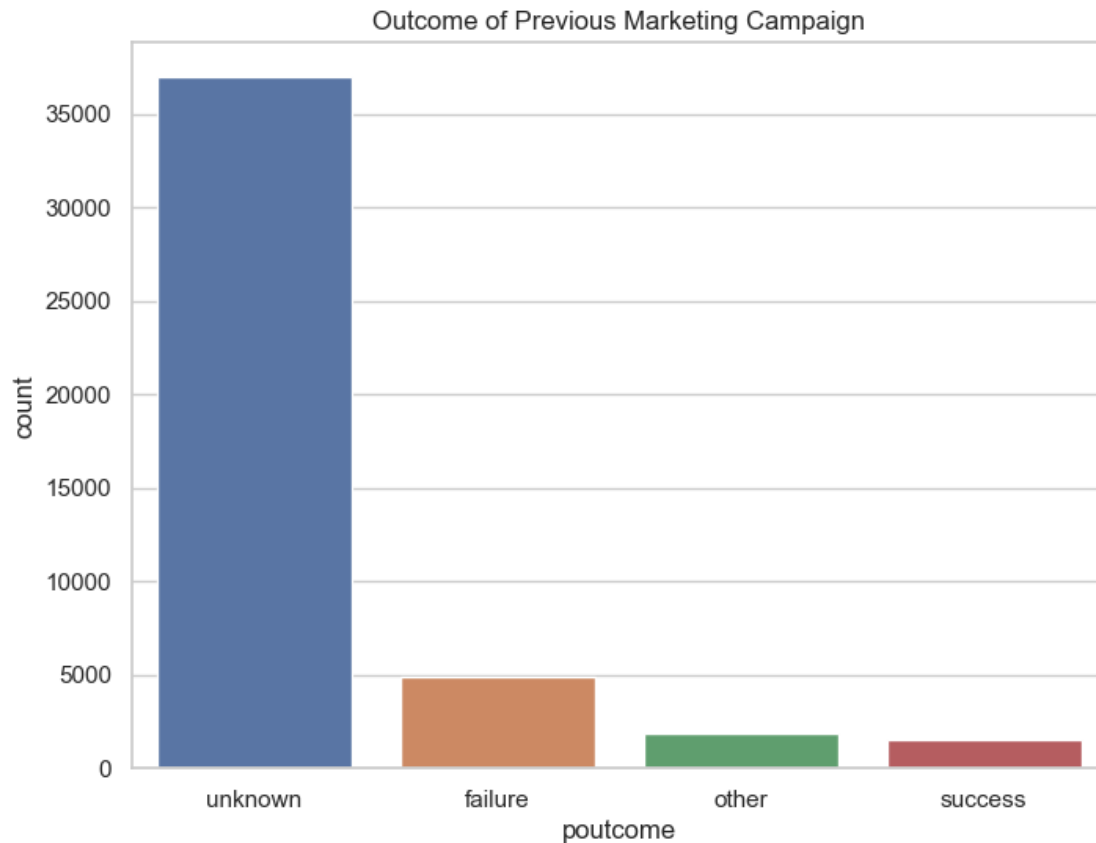
1.16 16. What were the outcomes of the previous marketing campaigns?

```
[39]: plt.figure(figsize=(8, 6))
sns.countplot(data=df, x='poutcome', palette='deep')
plt.title('Outcome of Previous Marketing Campaign')
plt.show()
```

/var/folders/h4/x04vtser10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/546613702.py:2:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='poutcome', palette='deep')
```

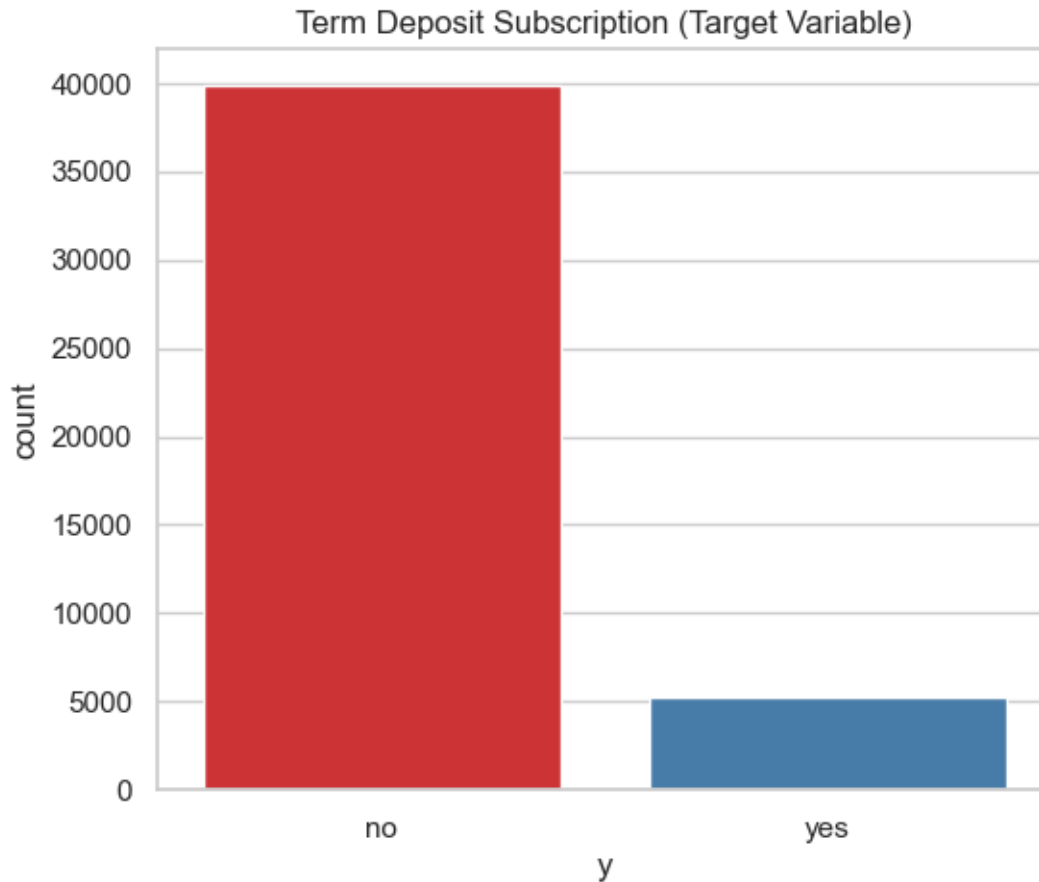
1.17 17. What is the distribution of clients who subscribed vs. those who did not?

```
[40]: plt.figure(figsize=(6, 5))
sns.countplot(data=df, x='y', palette='Set1')
plt.title('Term Deposit Subscription (Target Variable)')
plt.show()
```

/var/folders/h4/x04vtstr10h97rbk6bl8bq6pc0000gn/T/ipykernel_99482/3645651728.py:2
: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='y', palette='Set1')
```



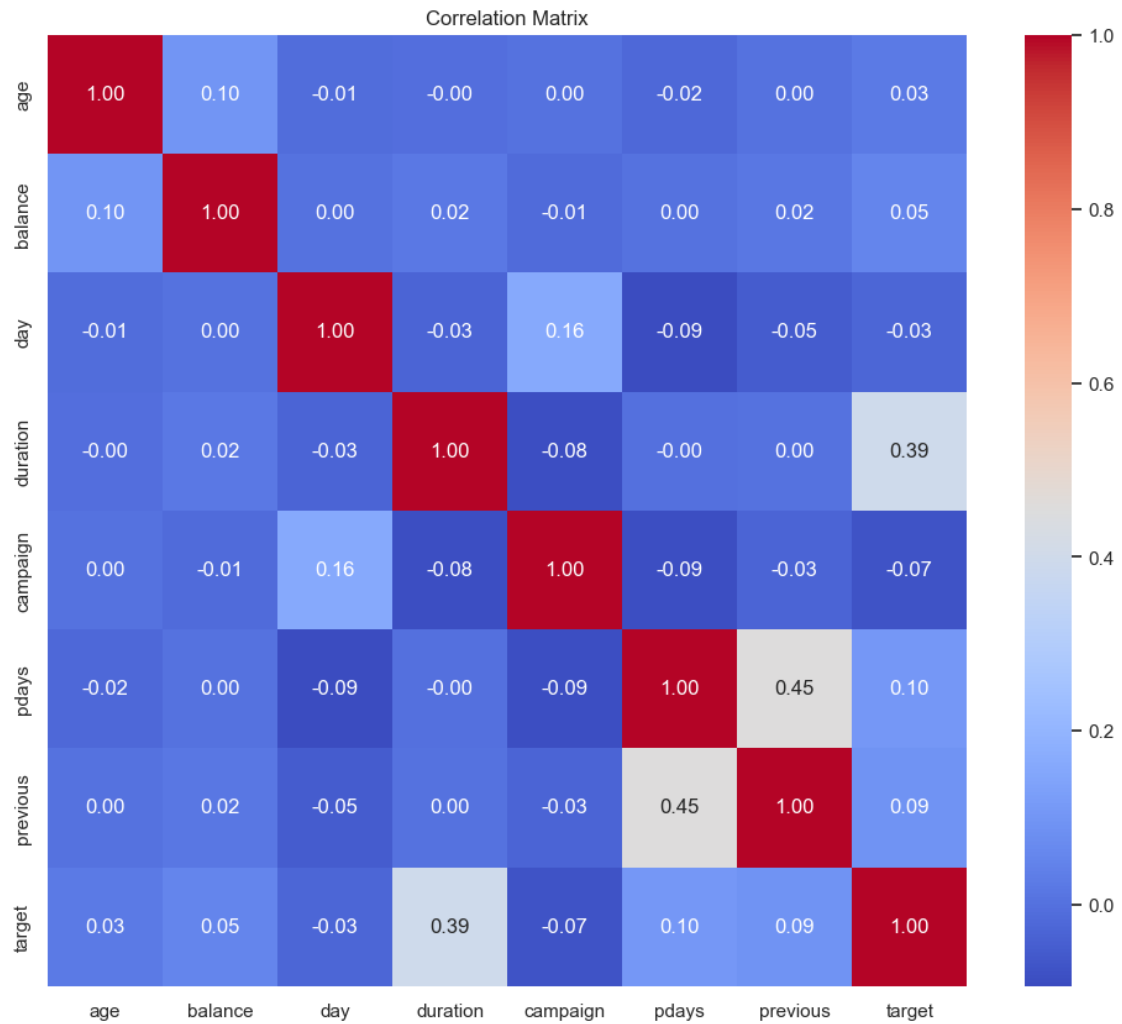
1.18 18. Are there any correlations between attributes and subscription likelihood?

```
[41]: # Convert target 'y' to numeric
df['target'] = df['y'].map({'yes': 1, 'no': 0})

# Select only numeric columns for correlation
numeric_df = df.select_dtypes(include=[np.number])

plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()

# Specifically check correlation with target
print(numeric_df.corr()['target'].sort_values(ascending=False))
```



```
target      1.000000
duration    0.394387
pdays      0.103699
previous    0.093576
balance     0.052821
age         0.025648
day         -0.028307
campaign    -0.073294
Name: target, dtype: float64
```